



Git Manual (v1.0)



What is Git?

Git is a **version control system** that tracks changes in your code over time. It allows you to:

- Save progress (commits)
 - Revert changes
 - Work on different features simultaneously (branches)
 - Collaborate with others using GitHub
-



Git Setup

Set your identity (one-time):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```



Starting a Project

Create a new Git repository:

```
git init
```

Initializes the current folder as a Git project.

Check the current status:

```
git status
```

Stage files for commit:

```
git add .
```

Adds all files in the current folder to the staging area.

Commit your changes:

```
git commit -m "Initial commit"
```

Creates a snapshot of your staged files.

Connecting to GitHub

Add a remote repository:

```
git remote add origin https://github.com/your-username/your-repo.git
```

Push to GitHub:

```
git push -u origin main
```

Pushes your code and sets **main** as the upstream branch.

Clone an existing GitHub repo:

```
git clone https://github.com/username/repo.git
```

Branching

Create and switch to a new branch:

```
git checkout -b feature-branch
```

Push the branch to GitHub:

```
git push origin feature-branch
```

List branches:

- Local branches:

```
git branch
```

- Remote branches:

```
git branch -r
```

- All branches:

```
git branch -a
```

Switch branches:

```
git checkout branch-name
```

Or in newer Git versions:

```
git switch branch-name
```

Delete branches:

- Local:

```
git branch -d branch-name
```

(use `-D` to force delete)

- Remote:

```
git push origin --delete branch-name
```

Merge a branch into another:

```
git checkout main  
git merge feature-branch
```

Merges `feature-branch` into `main`. Resolve conflicts if prompted.

Rebase a branch (advanced):

```
git checkout feature-branch  
git rebase main
```

Applies your changes on top of the latest `main` commits. Keeps history linear.

Stashing Changes

Temporarily save uncommitted work:

```
git stash
```

View stashed items:

```
git stash list
```

Apply the latest stash:

```
git stash apply
```

Apply and remove the latest stash:

```
git stash pop
```

Drop a specific stash:

```
git stash drop stash@{0}
```

Discarding Changes

Discard local (unstaged) changes:

```
git restore filename
```

Unstage a file:

```
git restore --staged filename
```

Discard all local changes:

```
git reset --hard HEAD
```

Remove untracked files:

```
git clean -f
```

Updating and Synchronizing

Pull latest changes from the remote:

```
git pull
```

Fetches and merges changes from the remote branch.

Fetch changes without merging:

```
git fetch origin
```

Updates your local references to the remote branch.

View differences:

- Uncommitted changes:

```
git diff
```

- Between commits:

```
git diff commit1 commit2
```

See who modified each line:

```
git blame filename
```

Revert a commit (safe undo):

```
git revert <commit-hash>
```

Creates a new commit that undoes the changes from the given commit.

Reset (careful!):

- Soft (keep changes staged):

```
git reset --soft HEAD~1
```

- Mixed (default, keep changes in working directory):

```
git reset --mixed HEAD~1
```

- Hard (delete all changes):

```
git reset --hard HEAD~1
```

Apply a commit from another branch:

```
git cherry-pick <commit-hash>
```

Useful for grabbing a specific fix or feature.

Viewing History

Show commit log:

```
git log
```

Show last commit:

```
git show
```

💡 Best Practices

- Use meaningful commit messages
 - Commit small, logical chunks
 - Pull latest changes before starting work
 - Never commit secrets (e.g. `.env` files)
 - Use `.gitignore` to exclude files from version control
-

📖 Glossary

- **Pull:** Fetches and merges changes from a remote branch
 - **Fetch:** Retrieves remote changes without applying them
 - **Diff:** Shows line-by-line changes between files or commits
 - **Blame:** Shows who made changes to each line in a file
 - **Revert:** Safely undo a commit by creating an opposite commit
 - **Reset:** Moves the HEAD pointer to a previous commit (can be dangerous)
 - **Cherry-pick:** Apply a specific commit from another branch
 - **Repository:** A Git-tracked project folder
 - **Commit:** A snapshot of code changes
 - **Branch:** A parallel line of development
 - **Merge:** Combining changes from one branch into another
 - **Rebase:** Rewriting commit history by applying changes on top of another base
 - **Stash:** A temporary shelf to save incomplete work
 - **Pull Request:** A request to merge code via GitHub
 - **Remote:** A GitHub-hosted copy of your project
-

Reference:

- [Git Documentation](#)
- [GitHub Docs](#)